MIT 8103 Portfolio Summary — ShopNaija Online Marketplace

Oct 13, 2026

Student: Oluwaleimu Oluwasegun · **Student ID:** 301865517 · **Programme:** Master of Information Technology, Miva Open University

Course: MIT 8103 Advanced Database Systems, Continuous Assessment, 2026/2027 First Semester · **Organisation:** ShopNaija Online Marketplace (simulated) · **GitHub (private):** github.com/segoslav/advanced-database-portfolio

1. Case Study

ShopNaija is a simulated Nigerian online marketplace selling electronics, fashion, home goods and groceries. Its database serves customers, administrators, inventory managers, finance staff and delivery agents. It stores customers, a catalogue whose attributes differ by category, stock, orders, payments, shipments and tracking events. The core operations are adding products, placing orders, updating stock, processing payments and tracking delivery. All five portfolios use this case study and one generated dataset of **5,000 customers, 2,000 products and 100,000 orders**.

2. Portfolio Activities

Portfolio 1 — Database Design and Modelling (PostgreSQL)

A 12-table schema was normalised from a flat order record (UNF → 3NF, checked against BCNF). Primary and foreign keys, `UNIQUE` and `CHECK` constraints (e.g. stock ⩾ 0), a partial unique index and `ON DELETE RESTRICT` enforce the business rules. Product attributes use `JSONB` because they differ by category. **11 constraint tests each rejected invalid data.**

Portfolio 2 — Query Processing and Optimisation

Eight queries were measured with `EXPLAIN (ANALYZE, BUFFERS)` before and after targeted composite, partial, expression and GIN indexes. Parcel tracking fell from **65.7 ms to 0.45 ms**, because PostgreSQL does not index foreign keys automatically. An index barely helped monthly revenue (159 → 103 ms) because the bottleneck was data volume; a materialised view cut it to **1.9 ms**.

Portfolio 3 — Transactions and Concurrency

A four-table place-order transaction demonstrated `COMMIT`, `ROLLBACK`, automatic abort and `SAVEPOINT`. Two concurrent sessions reproduced a **lost update (2 orders, 1 phone)**, which was fixed with `SELECT … FOR UPDATE`, optimistic version checks and `SERIALIZABLE` isolation. A non-repeatable read (`READ COMMITTED` vs `REPEATABLE READ`) and a deadlock were also demonstrated.

Portfolio 4 — NoSQL and Advanced Data Models (MongoDB)

The same data was remodelled as three MongoDB collections with embedded items, payments and tracking, plus a key-value cart with automatic expiry. `$jsonSchema` validation rejected bad data, and a single-document atomic update sold the last phone exactly once **without locks**. Aggregation matched PostgreSQL to the naira (2026 Electronics revenue **₦23,922,643,400** in both). The analysis recommends **polyglot persistence**: PostgreSQL for payments and stock, MongoDB for the catalogue and order tracking.

Portfolio 5 — Distributed and Cloud Databases

A three-node replica set in Docker replicated all 107,003 documents. Killing the primary triggered an election in **12.7 s**, with **45/45 orders saved and 0 lost**. A network partition showed the CAP trade-off: the majority side stayed writable, while the cut-off node was read-only and stale. `w: "majority"` was slower than `w: 3` because it waits for journaling. A multi-document transaction committed or aborted stock and order together. Shard-key analysis recommended **hashed** `customer_id` (a 33.2 / 33.6 / 33.1% split), because Lagos holds 2.9× an average state's orders.

3. Individual Technical Reflection

The design decision I'm most confident about is using JSONB to store product attributes instead of creating a separate column for every possible feature. Different products have different needs: a phone has storage and RAM, a dress has size and material, and rice has weight. Creating fixed columns for all these attributes would leave many fields empty. JSONB offered a more flexible solution while still allowing efficient searches through a GIN index. I applied similar thinking in Portfolio 4, where I recommended using different databases for different purposes. PostgreSQL handles payments and stock because these require strong constraints and reliable transactions across multiple rows. MongoDB, on the other hand, is better suited to the product catalogue and order tracking, where information is often accessed as complete documents.

Some of my biggest lessons came from the problems I encountered along the way. For example, my constraint-testing script reported an error even though the database had correctly prevented a delete operation. The issue was that PostgreSQL returned a different error name for the RESTRICT rule than my script expected. This taught me not to assume that every test failure means the system is broken. Sometimes, I need to examine the actual error message to understand what happened. In Portfolio 2, I also had to verify that my original execution plans had been preserved after creating indexes. Without a genuine baseline, comparing performance before and after optimisation would not have been reliable. I encountered another challenge in MongoDB, where an incorrectly written validation rule prevented the entire data-loading process because the rule was checked when the collection was created.

The distributed database tests helped me understand concepts that had previously seemed theoretical. Watching an order wait for 12.7 seconds while the replica set elected a new primary, without losing any orders, gave me a practical understanding of availability and retryable writes. I was also surprised to find that `w: "majority"` was slower than `w: 3`. Investigating this behaviour helped me understand how waiting for writes to be journaled to disk affects performance and durability, and why stronger guarantees can come at the cost of speed.

Looking back, I would approach a real-world project by agreeing on the data model and defining the evidence needed to validate the system before starting implementation. I would also commit smaller pieces of work more frequently throughout the project, rather than leaving too much until later. These experiences have taught me that good database design is not just about making things work; it is also about testing assumptions, understanding unexpected behaviour, and being able to demonstrate that the solution is reliable.

4. AI Use Declaration

Tool: Claude (Anthropic). **Used for:** planning; setup guidance; drafting scripts, READMEs and this summary; troubleshooting; filling README tables from my own outputs; and a sample reflection, which I rewrote in my own words. **Verification:** I ran every script on my own computer (PostgreSQL 18, MongoDB, Docker Desktop) and captured all outputs, timings and screenshots myself. I checked the results against expected behaviour (e.g. 11 PASS tests, matching SQL/MongoDB revenue, 0 orders lost in failover) and corrected the errors I found. A dated log is in `ai-use-log.md`.

5. References

- Data: all data is simulated, generated by `02_seed_data.sql`; no real personal data was used.
- dbdiagram.io (2026) *Database diagram tool*. https://dbdiagram.io
- Docker Inc. (2026) *Docker Compose documentation*. https://docs.docker.com/compose/
- Federal Republic of Nigeria (2023) *Nigeria Data Protection Act, 2023*.
- MongoDB Inc. (2026) *MongoDB Manual*. https://www.mongodb.com/docs/manual/
- PostgreSQL Global Development Group (2026) *PostgreSQL 18 Documentation*. https://www.postgresql.org/docs/18/

Appendix: Selected Evidence

The full evidence for each portfolio (scripts, saved outputs, execution plans and more than 30 screenshots) is in the repository's `results/` and `evidence/` folders. The work is recorded in progressive commits from 11 October 2026 onwards.

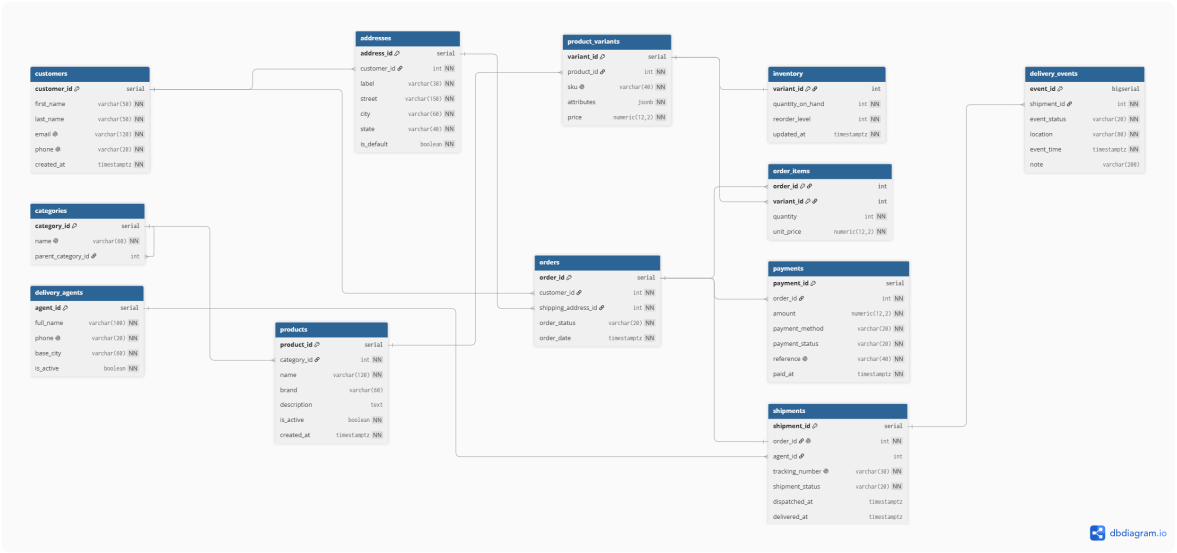


Figure 1: ShopNaija ER diagram (Portfolio 1)

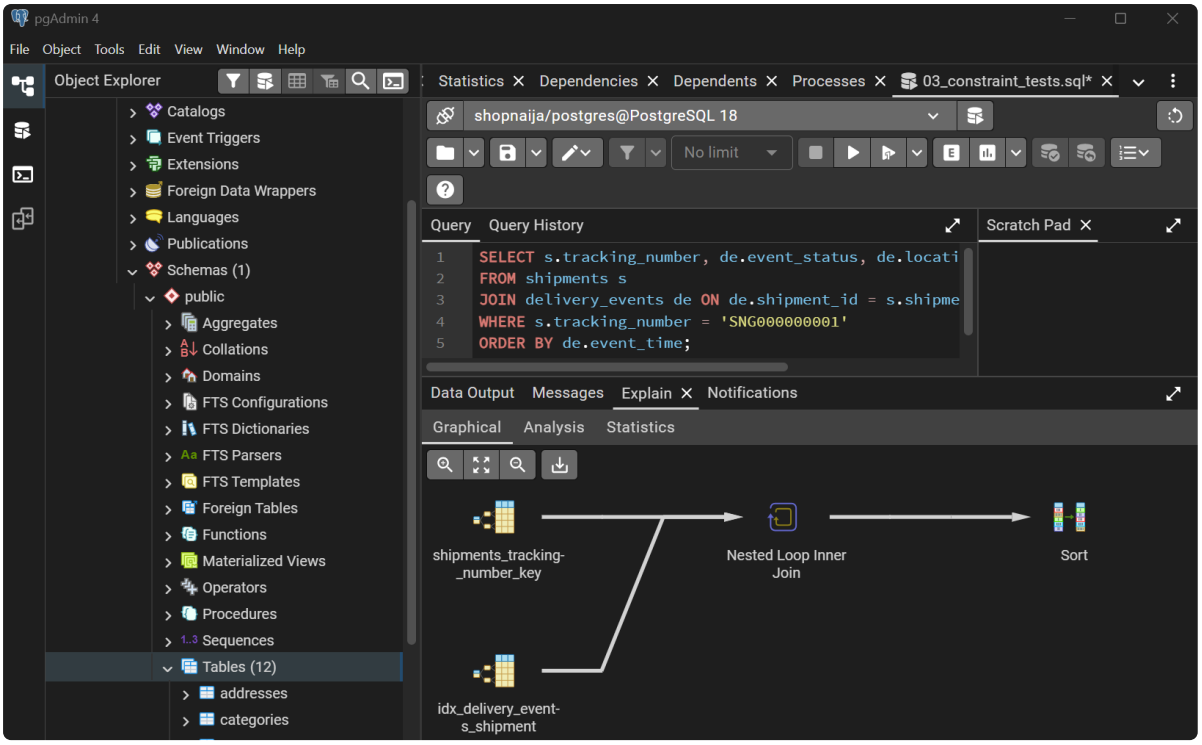


Figure 2: Parcel-tracking query after indexing: both tables now read through indexes, including the new `idx_delivery_events_shipment` (65.7 ms → 0.45 ms) (Portfolio 2)

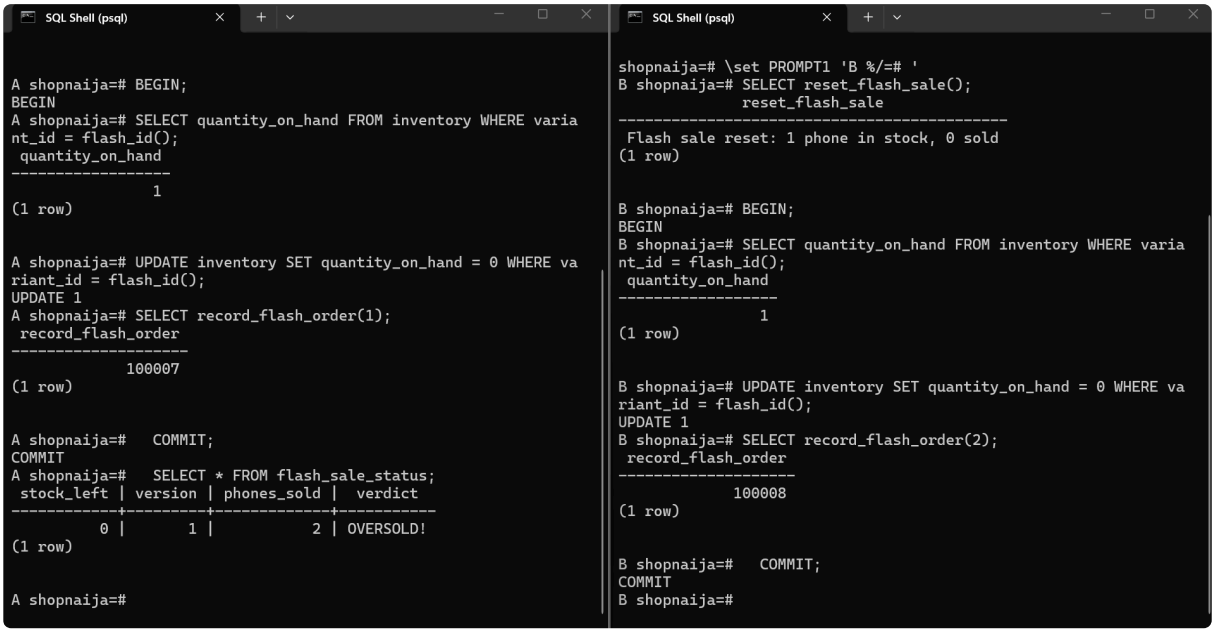


Figure 3: Lost update: two sessions both buy the last phone, oversold (Portfolio 3)

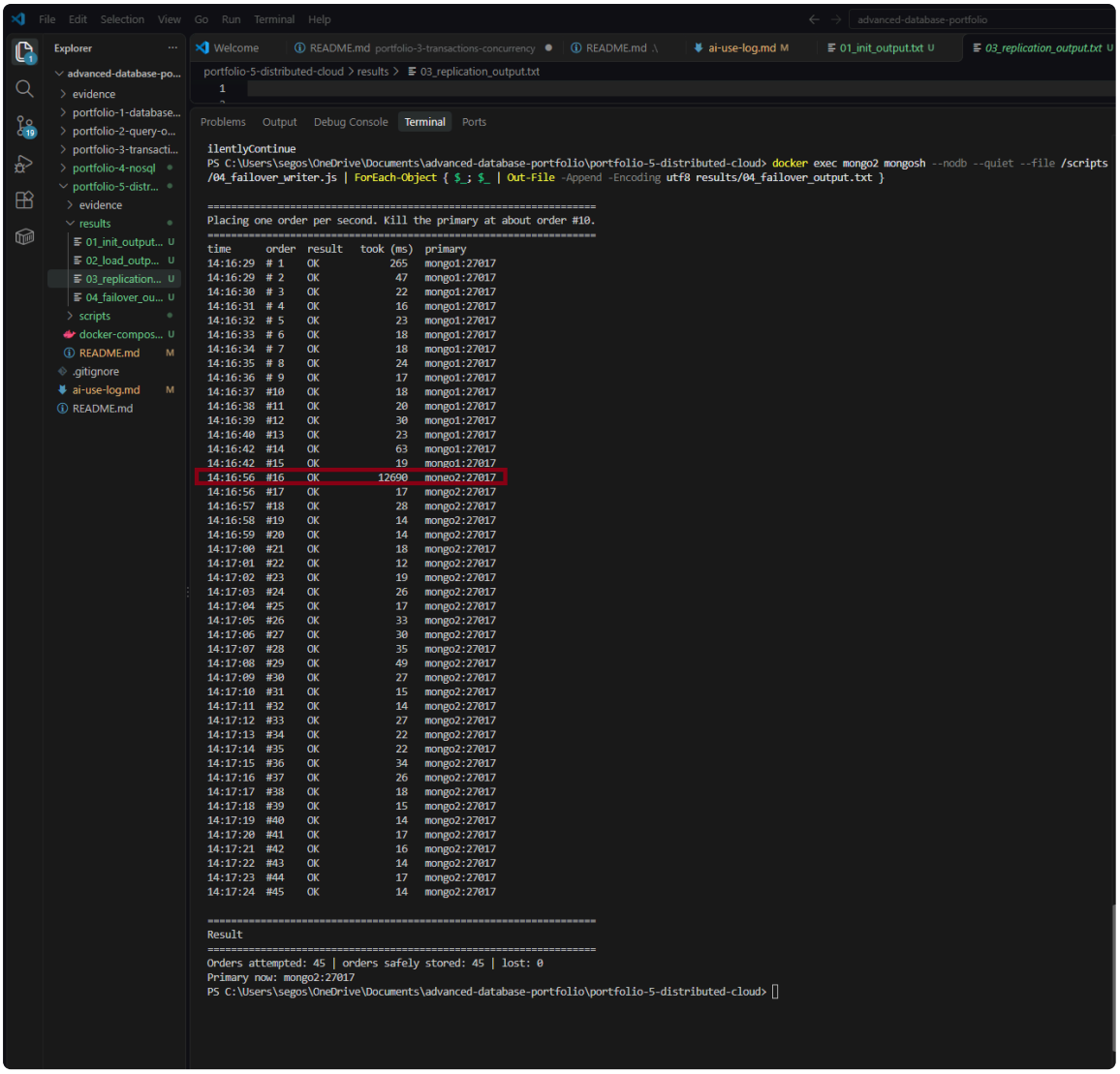


Figure 4: Primary killed at order #16; election pause of 12.7 s, 45/45 orders saved (Portfolio 5)